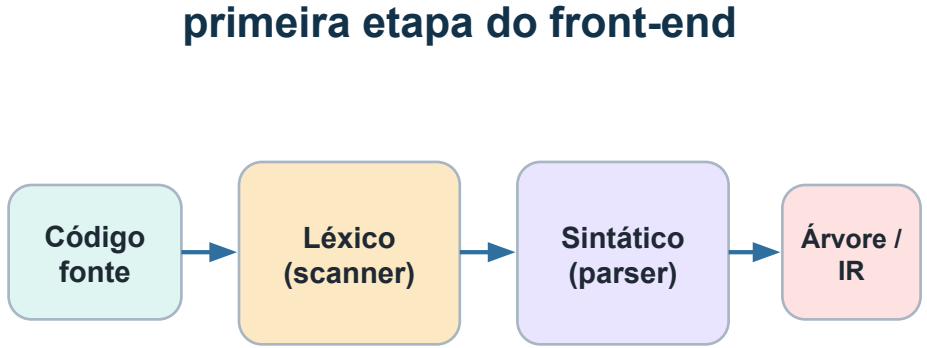
Análise léxica em compiladores

Nesta aula, vamos estudar como o compilador transforma texto bruto em uma sequência organizada de tokens. A análise léxica é a ponte entre caracteres e estruturas sintáticas, usando padrões, expressões regulares e autômatos para reconhecer identificadores, números, operadores, palavras reservadas e erros.



Objetivos de aprendizagem

O QUE O ALUNO DEVERÁ CONSEGUIR FAZER

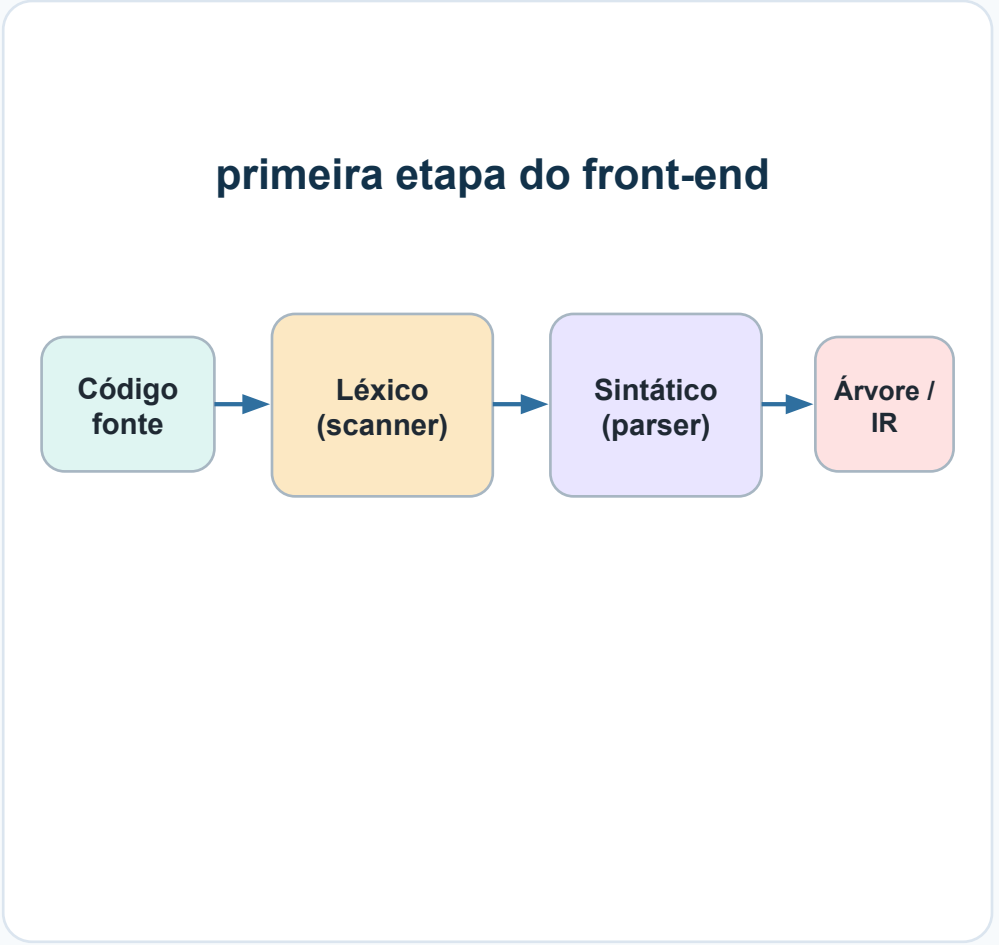
Ao final da aula, o aluno deverá diferenciar token, lexema e padrão; representar regras léxicas por expressões regulares; relacionar padrões a autômatos finitos; compreender estratégias de tratamento de erros; e construir uma implementação simples de scanner para uma linguagem didática.



Onde a análise léxica entra no compilador?

PRIMEIRA ETAPA DO FRONT-END

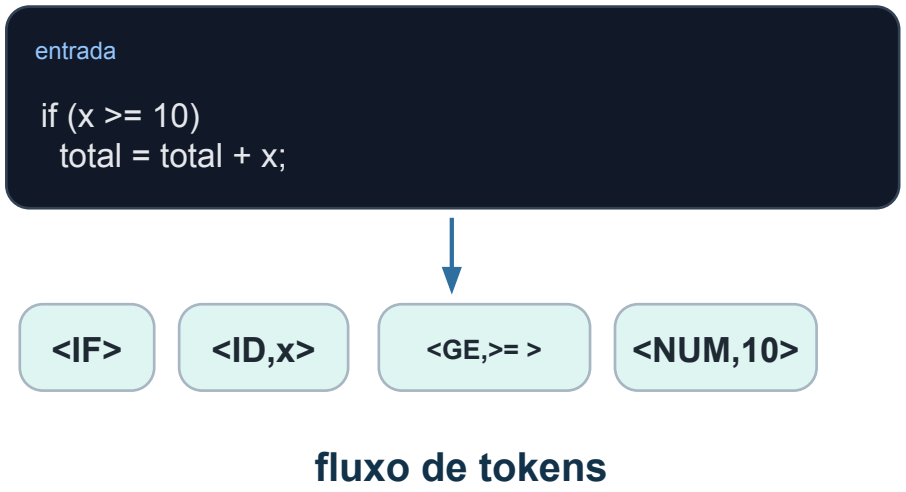
A análise léxica geralmente aparece no início do front-end do compilador. Ela recebe o código fonte como sequência de caracteres, remove partes irrelevantes para a gramática, como espaços e comentários, reconhece lexemas e entrega tokens para a análise sintática trabalhar com uma entrada mais organizada.



Da sequência de caracteres aos tokens

TRANSFORMAÇÃO DA ENTRADA

Um programa é gravado como texto, mas o compilador precisa identificar unidades com significado. A sequência “total = 10 + x” não é tratada como uma frase comum: ela pode ser separada em identificador, operador de atribuição, número, operador de soma e outro identificador.

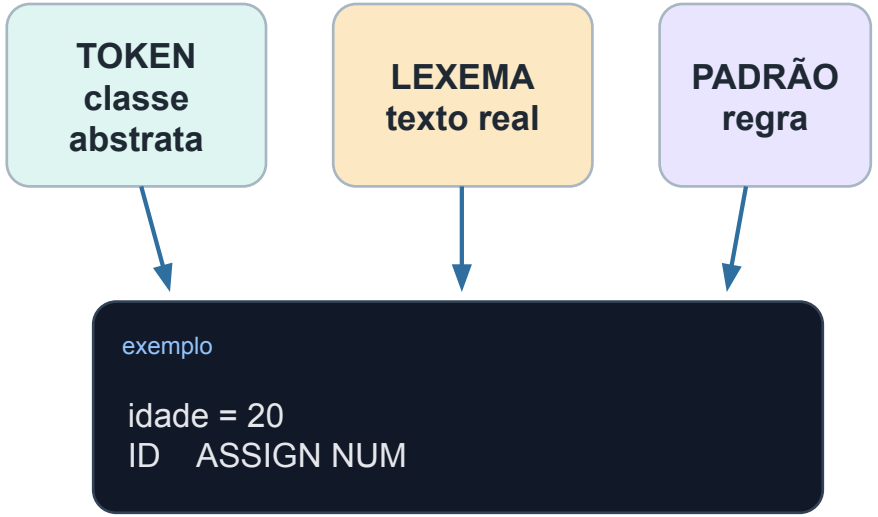


fluxo de tokens

Token, lexema e padrão

TRÊS CONCEITOS FUNDAMENTAIS

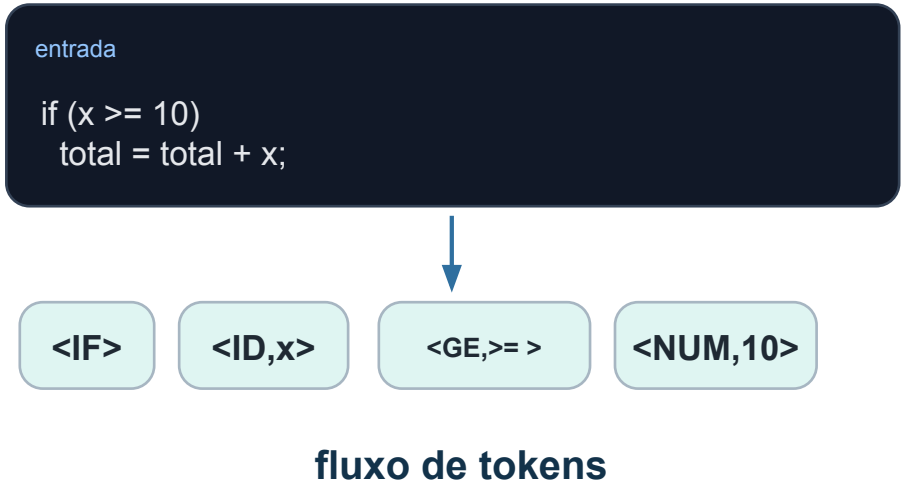
Token é a categoria produzida pelo analisador léxico, lexema é o texto concreto encontrado na entrada, e padrão é a regra que descreve quais lexemas pertencem ao token. Por exemplo, em “idade = 20”, idade é lexema de identificador, e 20 é lexema de número.



Exemplo completo de tokenização

OBSERVANDO UMA LINHA DE CÓDIGO

Considere a instrução “if (x >= 10) total = total + x;”. O scanner reconhece a palavra reservada if, parênteses, identificadores, operador relacional, número, atribuição, soma e ponto e vírgula. O resultado é uma lista ordenada de tokens, preservando a posição de cada lexema.

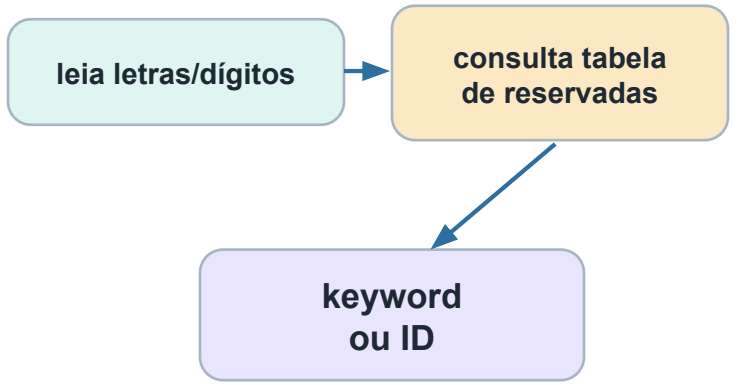


fluxo de tokens

Palavras reservadas e identificadores

QUANDO TEXTOS PARECIDOS TÊM CATEGORIAS DIFERENTES

Identificadores e palavras reservadas costumam seguir padrões parecidos, pois ambos podem ser formados por letras. A diferença aparece depois da leitura do lexema: se o texto estiver em uma tabela de palavras reservadas, o token emitido será keyword; caso contrário, será identificador.




```
tabela
if  → IF
int → TYPE
valor→ ID
```

Espaços, quebras de linha e comentários

O QUE É IGNORADO TAMBÉM PRECISA SER RECONHECIDO

Nem tudo que o scanner lê vira token para o parser. Espaços, tabulações, quebras de linha e comentários geralmente são descartados ou enviados para um canal especial. Ainda assim, eles precisam ser reconhecidos para atualizar linha, coluna e localização de erros.



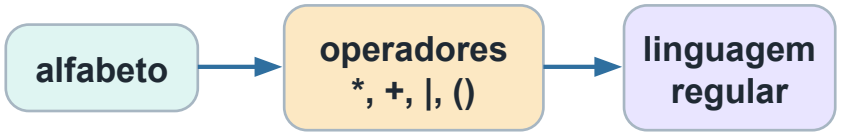
Padrões léxicos e expressões regulares

COMO DESCREVEMOS OS LEXEMAS VÁLIDOS

Expressões regulares descrevem conjuntos de strings e são muito usadas para especificar tokens. Um identificador pode começar com letra ou sublinhado e continuar com letras, dígitos ou sublinhado. Um número pode ter uma parte inteira e, opcionalmente, uma parte decimal.

padrões léxicos

```
ID  = [A-Za-z_][A-Za-z0-9_]*
NUM = [0-9]+(\.[0-9]+)?
WS  = [ \t\n]+
```

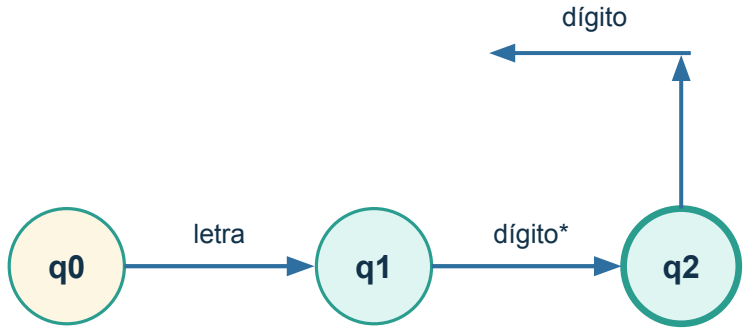


Sugestão: documentação de expressões regulares da MDN

Autômatos finitos como reconhecedores

DO PADRÃO AO COMPORTAMENTO DE LEITURA

Um autômato finito pode reconhecer se uma sequência pertence ao padrão de um token. Cada estado representa uma situação parcial da leitura, e cada transição consome caracteres. Quando a leitura para em um estado final, o scanner possui evidência para emitir um token.



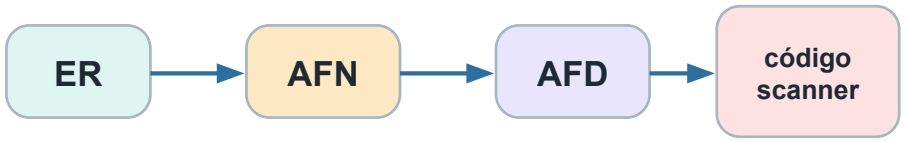
reconhecedor de ID

AFN, AFD e determinização

POR QUE O SCANNER PREFERE AFD

Expressões regulares podem ser convertidas em autômatos não determinísticos e, depois, em autômatos determinísticos. Na execução de um scanner, o AFD é conveniente porque, para cada estado e caractere, existe uma próxima ação bem definida, o que torna a leitura rápida e previsível.

Material prático: JFLAP



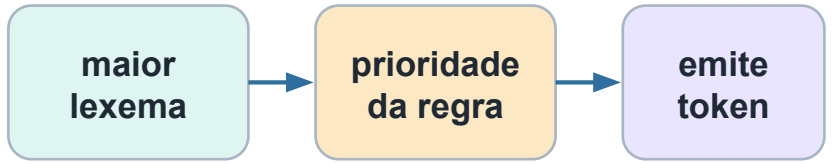
ideia da determinização

```
closure({q0})
move(S, c)
novo estado = conjunto
```

Regra do maior lexema e prioridade

COMO RESOLVER AMBIGUIDADES

Quando mais de um token pode começar no mesmo ponto, o scanner normalmente escolhe o maior lexema possível. Se ainda houver empate, aplica a ordem de prioridade das regras. Assim, “>=” deve ser reconhecido antes de separar “>” e “=”, preservando o significado correto.



exemplo

```
entrada: >=
GE : >=
GT : >
ASSIGN : =
```

Fluxo básico do analisador léxico

ALGORITMO CONCEITUAL

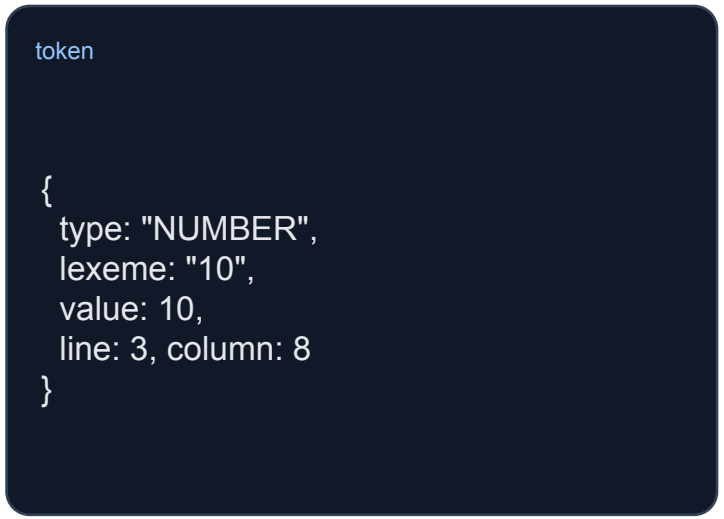
O scanner mantém uma posição atual na entrada, tenta avançar enquanto encontra transições válidas e memoriza o último estado final alcançado. Quando não consegue avançar, ele emite o token correspondente ao último final ou relata erro se nenhum padrão foi reconhecido.



Estrutura de um token na prática

MAIS DO QUE APENAS O NOME DA CATEGORIA

Em uma implementação real, o token costuma guardar tipo, lexema, valor literal e posição no arquivo. O valor literal converte “10” em número inteiro, enquanto linha e coluna permitem mensagens de erro precisas. Essas informações acompanham o token até fases posteriores.

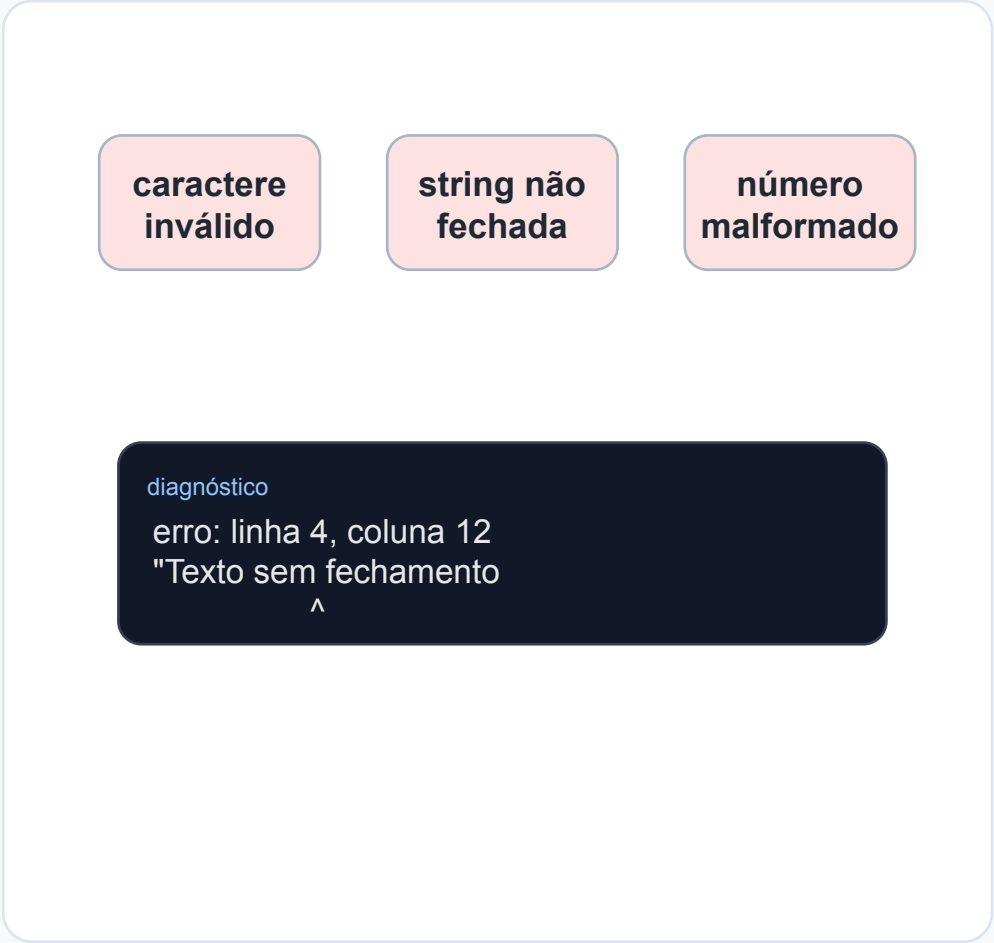


metadados permitem mensagens melhores

Tratamento de erros léxicos

QUANDO A ENTRADA NÃO COMBINA COM NENHUM PADRÃO

Erros léxicos ocorrem quando o scanner encontra símbolos inesperados, strings não fechadas, números malformados ou caracteres fora do alfabeto permitido. O ideal é informar claramente o local, mostrar o trecho problemático e continuar quando for seguro para encontrar outros erros.



Recuperação de erros

CONTINUAR A ANÁLISE SEM CONFUNDIR TUDO

Depois de detectar um erro, o scanner pode criar um token especial de erro, descartar caracteres até um ponto de sincronização ou avançar apenas um caractere. A estratégia escolhida deve reduzir mensagens em cascata e preservar o máximo possível da análise restante.



recuperação simples

```
while c not in {; ,
 EOF}:
   advance()
```

Implementação manual de um scanner

CONTROLE DIRETO DA LEITURA

Uma implementação manual percorre a entrada com variáveis como start, current, line e column. Funções auxiliares verificam o próximo caractere, avançam a posição, consomem comentários, reconhecem números e identificadores, e adicionam tokens a uma lista de saída.

laço principal

```
while not end:
  start = current
  c = advance()
  scanToken(c)
return tokens
```

start

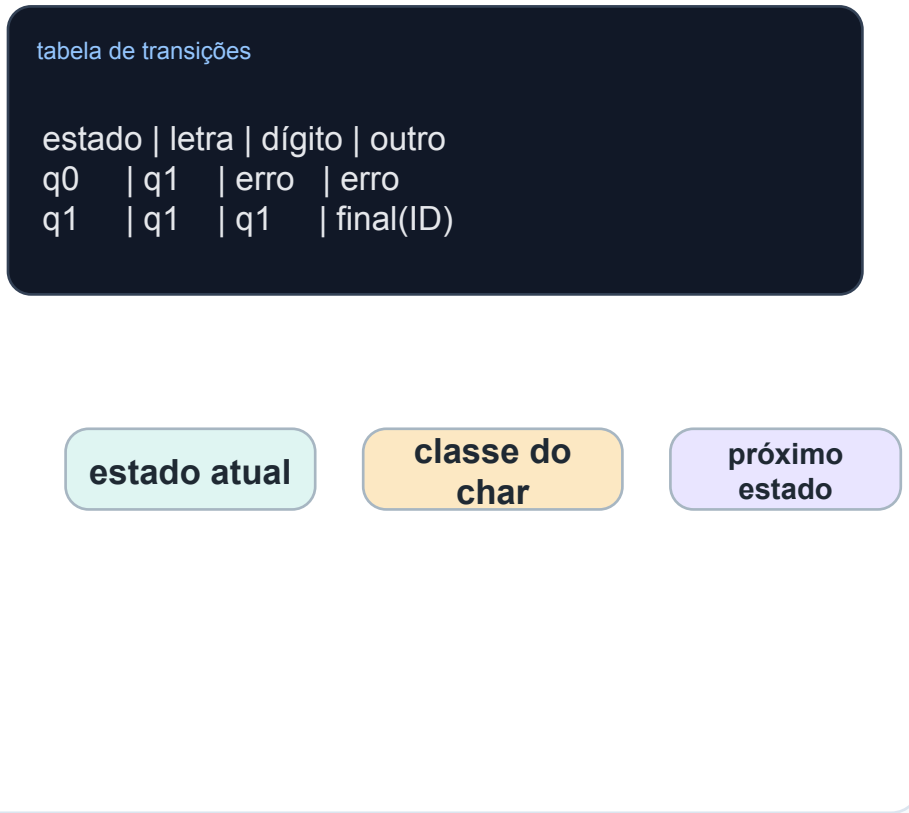
current

linha

Implementação com tabela de transições

EXECUTANDO UM AUTÔMATO DETERMINÍSTICO

Outra abordagem é representar o AFD em uma tabela. A linha indica o estado atual e a coluna indica a classe do caractere lido. O valor da célula informa o próximo estado ou uma ação final. Essa técnica aproxima a implementação da teoria de autômatos.



Ferramentas geradoras de analisadores léxicos

FLEX, ANTLR E ALTERNATIVAS ACADÊMICAS

Ferramentas como Flex, ANTLR e JFlex permitem declarar padrões e ações para gerar automaticamente partes do analisador léxico. Elas são úteis em projetos maiores, pois reduzem código repetitivo, centralizam as regras da linguagem e integram o scanner com o analisador sintático.



regra estilo ANTLR

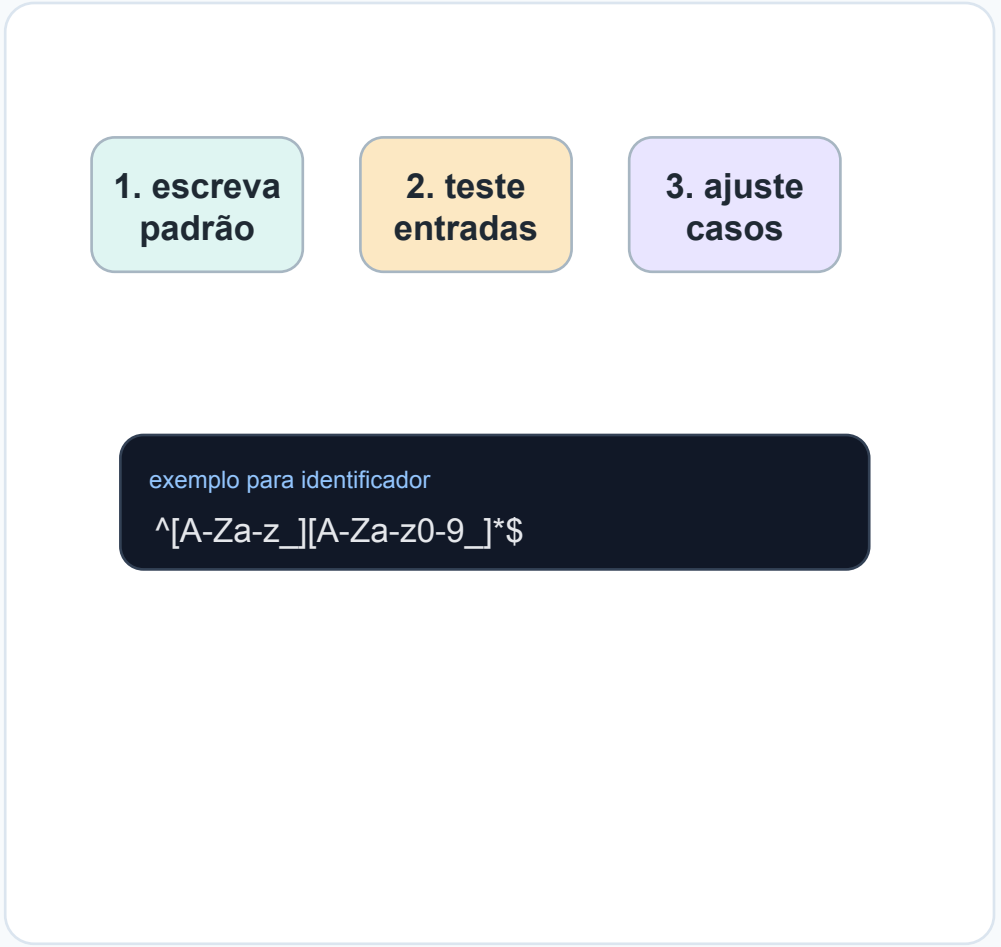
```
ID  : [a-zA-Z_][a-zA-Z0-9_]* ;
INT : [0-9]+ ;
WS  : [ \t\r\n]+ -> skip ;
```

Flex manual e ANTLR docs: westes.github.io/flex/manual/ | antlr.org

Demonstração prática: testando padrões

ATIVIDADE RÁPIDA COM REGEX ONLINE

Uma forma simples de praticar é escrever padrões de tokens em uma ferramenta gratuita de teste de expressões regulares. O aluno pode testar entradas válidas e inválidas, observar onde a expressão casa, ajustar limites e perceber casos ambíguos antes de implementar o scanner.

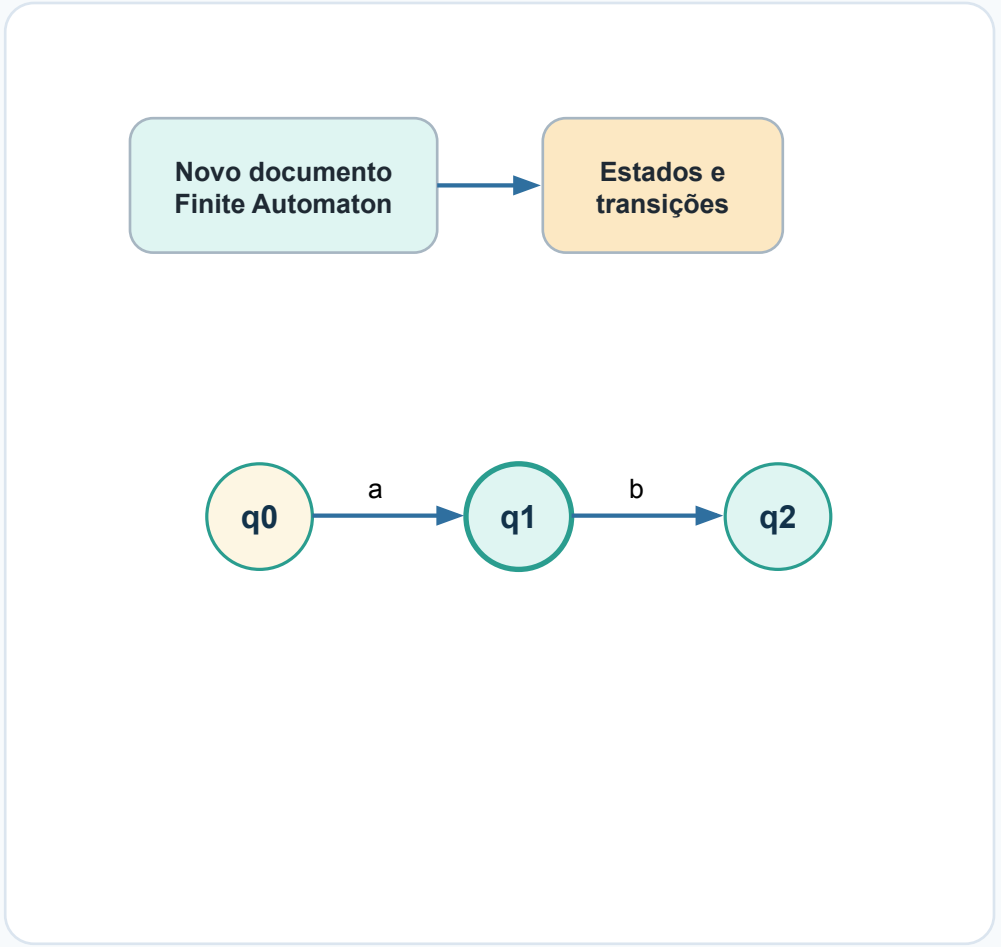


Ferramenta gratuita sugerida: regex101.com

Demonstração prática: autômatos no JFLAP

VISUALIZANDO ESTADOS E SIMULAÇÃO

No JFLAP, é possível criar estados, marcar estado inicial e final, desenhar transições e simular entradas. Para a análise léxica, essa visualização ajuda a entender como um padrão regular se transforma em comportamento de leitura, principalmente em identificadores e números.



JFLAP é indicado no material anexo como ferramenta gratuita: jflap.org

Estudo de caso: linguagem CALC

UM CONJUNTO PEQUENO DE TOKENS

Imagine uma linguagem didática chamada CALC, com declarações simples, atribuições e expressões aritméticas. O scanner precisa reconhecer palavras como let, identificadores, números inteiros ou decimais, operadores +, -, *, /, parênteses, sinal de atribuição e ponto e vírgula.

entrada CALC

```
let soma = 10 + valor;
```

saída léxica

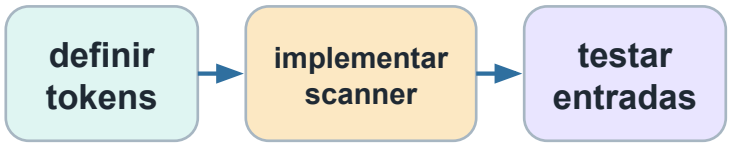
```
LET ID ASSIGN NUM PLUS ID SEMI
```

gramática fica mais limpa

Passo a passo de implementação

COMO CONSTRUIR O SCANNER DA CALC

Primeiro, liste todos os tipos de tokens. Segundo, escreva exemplos de lexemas válidos e inválidos. Terceiro, implemente funções para avançar, verificar próximo caractere e adicionar tokens. Quarto, trate espaços, comentários e erros. Por fim, crie testes com entradas pequenas.




```
teste esperado
scan("x=10")
→ ID ASSIGN NUM EOF
```

Exemplo de scanner manual

TRECHO DE PSEUDOCÓDIGO DIDÁTICO

O pseudocódigo do scanner normalmente possui um laço principal que avança até o fim da entrada. Em cada iteração, ele define o início do próximo lexema, lê um caractere e chama uma rotina que decide se aquele caractere inicia operador, número, identificador, comentário ou erro.

laço principal

```
while not end:
  start = current
  c = advance()
  scanToken(c)
return tokens
```

start

current

linha

Checklist de qualidade do lexer

COMO AVALIAR SE A IMPLEMENTAÇÃO ESTÁ BOA

Um bom analisador léxico reconhece todos os tokens previstos, ignora corretamente espaços e comentários, produz posições confiáveis, diferencia reservadas de identificadores, resolve operadores ambíguos pelo maior lexema, apresenta erros claros e possui testes com casos positivos, negativos e limites.



Relação com a análise sintática

POR QUE O PARSER DEPENDE DO LEXER

O parser trabalha melhor quando recebe tokens consistentes. Em vez de ler cada caractere, ele reconhece estruturas como comandos, expressões e blocos a partir de categorias léxicas. Uma tokenização errada pode gerar erros sintáticos confusos, mesmo que a gramática esteja correta.

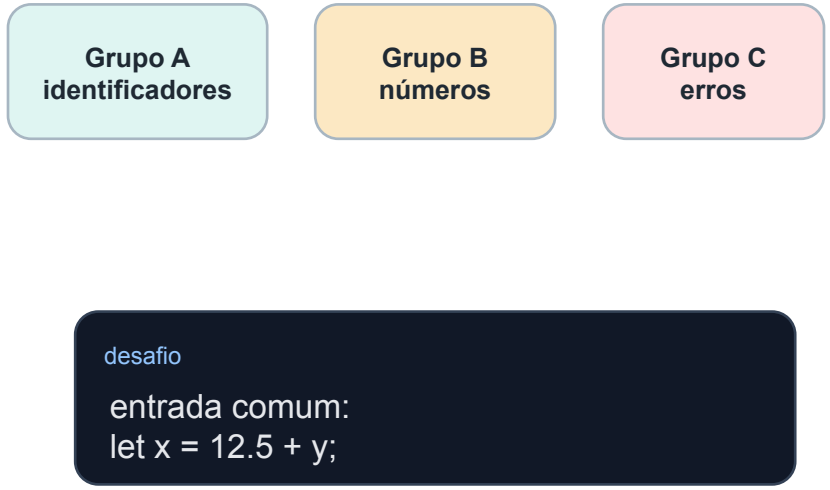



```
interface
if ( expr ) stmt
IF LPAREN ID RPAREN ID
```

Atividade prática sugerida

CONSTRUIR, TESTAR E EXPLICAR

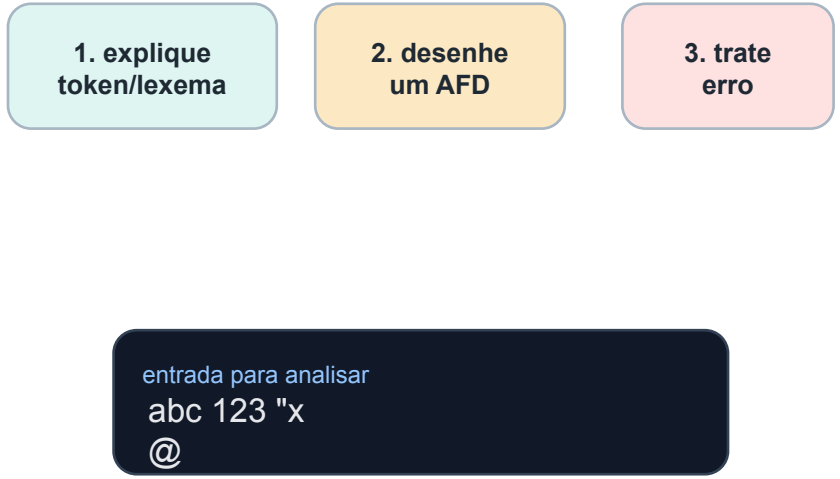
Em grupos, os alunos devem definir os tokens da linguagem CALC, escrever padrões para cada token, testar entradas em uma ferramenta de regex e implementar uma primeira versão do scanner. Ao final, cada grupo deve apresentar um erro encontrado e explicar como foi tratado.



Avaliação de aprendizagem

QUESTÕES PARA FIXAÇÃO

Explique a diferença entre token, lexema e padrão usando um exemplo próprio. Desenhe um AFD simples para identificadores. Tokenize uma linha de código com operadores compostos. Descreva uma estratégia para tratar string não fechada e apresente uma mensagem de erro adequada.



Avaliação e referências

Avaliação sugerida: implementar um scanner simples para a linguagem CALC, entregar tabela de tokens, exemplos de entradas válidas e inválidas, e explicar pelo menos uma estratégia de tratamento de erro.

Referências: AHO, A. V. et al. Compilers: Principles, Techniques, and Tools. Pearson. RODGER, S. H.; FINLEY, T. W. JFLAP – An Interactive Formal Languages and Automata Package, 2005. JFLAP: https://www.jflap.org/. Flex Manual: https://westes.github.io/flex/manual/. ANTLR Lexer Rules: https://github.com/antlr/antlr4/blob/dev/doc/lexer-rules.md. Crafting Interpreters – Scanning: https://craftinginterpreters.com/scanning.html. MDN Regular Expressions: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide/Regular_expressions.